

Introducción a los modelos de ciclos de vida

El ciclo de vida del desarrollo de software

El proceso de desarrollo del software involucra llevar a cabo un conjunto de actividades para su producción. Este proceso se denomina ciclo de vida porque abarca desde que surge la necesidad inicial de un sistema hasta que el mismo sale de uso.

Existen varios modelos de ciclos de vida, también llamados paradigmas de proceso, que tienen que ver con la forma específica en que se realizan las tareas y la secuencia de actividades que involucran. El ciclo de vida busca establecer el orden en que un proyecto se especifica, se prototipea, diseña, prueba e implementa, por ejemplo.

Dentro del desarrollo de un proyecto es muy importante seleccionar el ciclo de vida que se empleará, ya que una elección correcta ayudará al cumplimiento de los objetivos, como productividad en el desarrollo, calidad del producto, disminución de riesgos, o un mejor acercamiento al cliente.

Abordaremos cinco modelos de ciclos de vida:

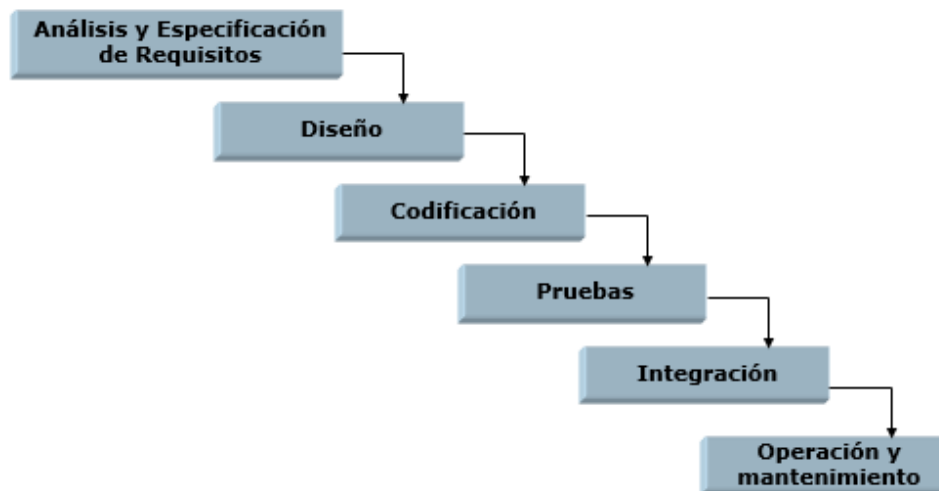
- Ciclo de vida clásico
- Modelo espiral
- Modelo incremental
- Prototipos
- Modelo evolutivo UP

Ciclo de vida clásico

Es el primer modelo de ciclo de vida, pero se sigue empleando, y fue la base para muchos de los modelos que le siguieron. Por algunos autores también es llamado modelo de cascada o lineal debido al enfoque que plantea, donde las actividades se dan en forma secuencial y cada etapa debe estar completamente finalizada para habilitar el inicio de la siguiente, lo que lo transforma en un modelo muy rígido de seguir. Está conformado por las siguientes actividades: análisis y especificación de requerimientos, diseño, codificación, pruebas, integración, operación y mantenimiento.

Cabe aclarar que el nombre de las etapas puede variar en diferentes bibliografías, por ejemplo, se puede encontrar que la etapa de prueba esté separada en prueba unitaria y pruebas de integración.

Puede observarse su gráfica en la figura que se encuentra a continuación:



Estas etapas del modelo conforman las actividades a desarrollarse:

Análisis y especificación de requerimientos: en esta etapa, se buscará relevar los requerimientos del sistema, sus características, normas, políticas y procedimientos. Luego, se llevará a cabo un análisis de la información para concluir en una especificación formal del sistema. La fase de análisis está asociada a determinar “qué” se debe hacer.

Diseño: aquí nos ocuparemos del “cómo”. Para disminuir la complejidad del sistema, en general, se lo suele dividir en piezas más pequeñas denominadas módulos. Los autores Gane y Sarson dicen que “el diseño es el proceso (iterativo) de tomar un modelo lógico de un sistema con un conjunto de objetivos fuertemente establecidos para este sistema, y producir la especificación de un sistema físico que satisfaga dichos objetivos”.

En la etapa de diseño deberá trabajarse para describir los algoritmos que se emplearán en cada módulo; la estructura de datos a utilizar; la arquitectura del software; el detalle de cómo llevar a cabo cada procedimiento, así como las interfaces del sistema.

Codificación: se entrega al equipo de programación lo que se especificó en el diseño para que lo traduzcan al lenguaje de programación que se haya definido para el sistema a desarrollar.

Pruebas: en forma posterior a la codificación, debe constatarse que el código generado responda a los requerimientos que fueron establecidos y, así, las entradas que se introduzcan generen las salidas que se esperan.

Integración: los módulos, o componentes del sistema, se combinan (integran) para conformar el sistema de información.

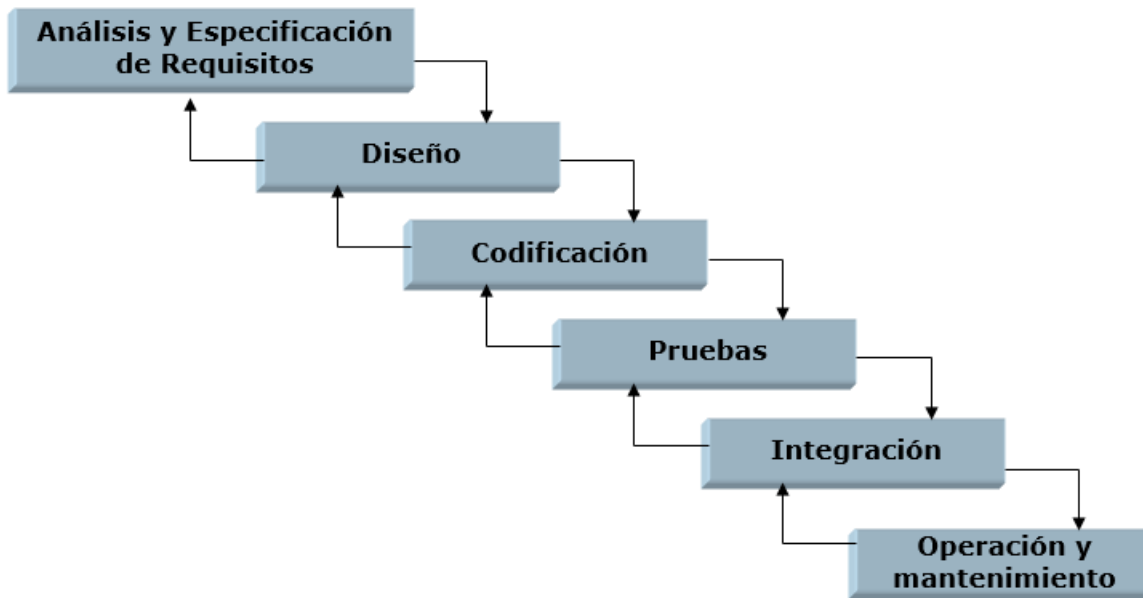
Operación y mantenimiento: la operación corresponde a la puesta en funcionamiento del sistema, su uso por parte del cliente y/o usuarios. El mantenimiento refiere a cualquier modificación que deba realizarse al sistema luego de su entrega al cliente, lo cual puede ser, por ejemplo: para corregir algún error o falla no detectada en la etapa de prueba, adaptar al sistema a algún cambio que se produjo en el entorno, o para incorporar una nueva funcionalidad al mismo.

Como indicábamos previamente, el modelo refleja un desarrollo marcado por la sucesión escalonada de las etapas que lo componen, por lo cual no requiere estar a cargo de personal muy experto en su administración ni para su planificación porque no habrá solapamiento de actividades. Esto implica que todo el grupo se encuentra en la misma etapa del desarrollo en un momento dado.

Por tratarse de un modelo extremadamente rígido, debería aplicarse a desarrollos donde los requisitos se encuentren completamente definidos al inicio del proyecto y no se esperen cambios. Si bien permite confeccionar un cronograma para el desarrollo, no le permite al cliente ver los avances hacia el producto hasta etapas muy avanzadas, como ser en las pruebas. Por esto es necesario comprender muy bien los requerimientos si se selecciona este modelo.

El paso del tiempo hizo que en el mismo modelo se incluyera el retorno a etapas anteriores debido a que, en ocasiones es posible que se necesite volver para corregir algún error producido anteriormente, lo cual lo transforma en un modelo más amoldado a la realidad de lo que puede suceder en la práctica.

El modelo modificado se presenta de la siguiente figura:

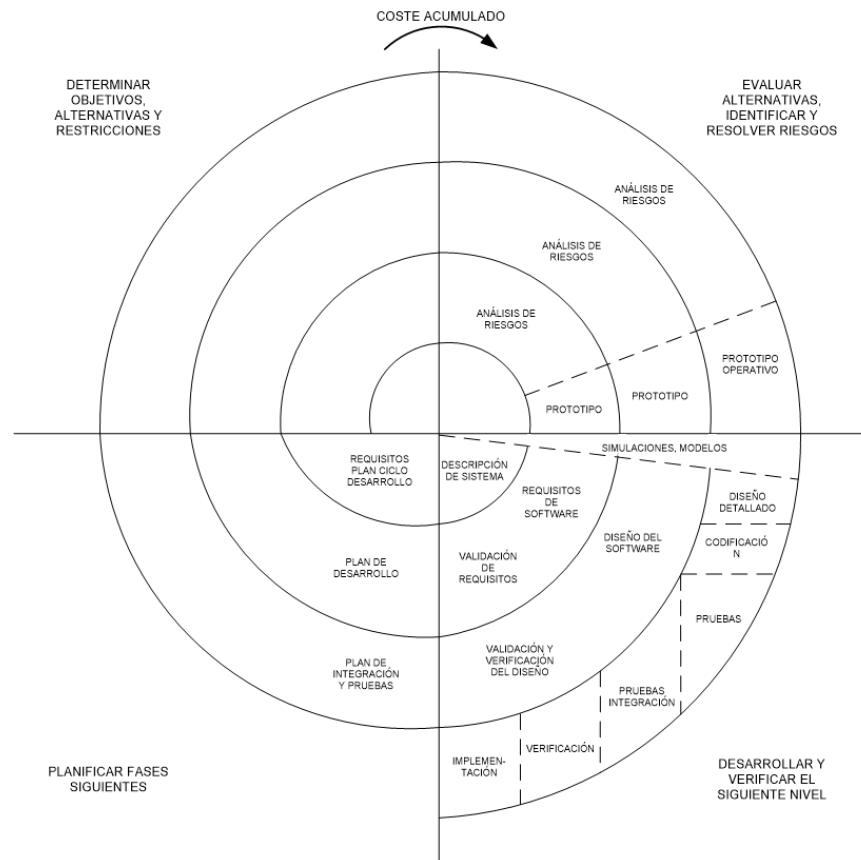


Es importante destacar que para dar por culminada una etapa y poder pasar a la siguiente, es necesario completar toda la documentación de dicha etapa. Esto ocasiona que se finalice con un gran volumen de documentación el proyecto.

Modelo espiral

El modelo en espiral surge en los 80. Fue propuesto originalmente por Barry Boehm, un reconocido profesional quien ha realizado múltiples aportes al área de la ingeniería informática. Este modelo plantea un cambio importante, ya que lleva a cabo la incorporación del análisis de riesgos al ciclo de vida del proyecto que, hasta el momento, no estaba contemplado. Se plantea al proyecto como una sumatoria de mini proyectos, cada uno centrado en uno o más riesgos.

Veamos en la siguiente figura la gráfica del modelo:



Los cuatro cuadrantes representan cuatro actividades por las que pasa el desarrollo, a saber:

- **Planificación:** se deben definir los objetivos específicos y alternativas, teniendo en cuenta las restricciones del proceso y del producto. Se decide qué parte del desarrollo se abordará en el ciclo.
- **Análisis de riesgos:** se lleva a cabo un análisis de alternativas, se seleccionan la más ventajosa, y se identifican y previenen los posibles riesgos.
- **Ingeniería:** se lleva a cabo el desarrollo del producto, basado en la planificación inicial y el resultado del análisis de riesgos. Se aplica algún modelo, como por ejemplo, el de cascada que vimos previamente.
- **Evaluación del cliente:** este debe llevar a cabo una valoración de lo realizado para determinar cómo se procede a continuación.

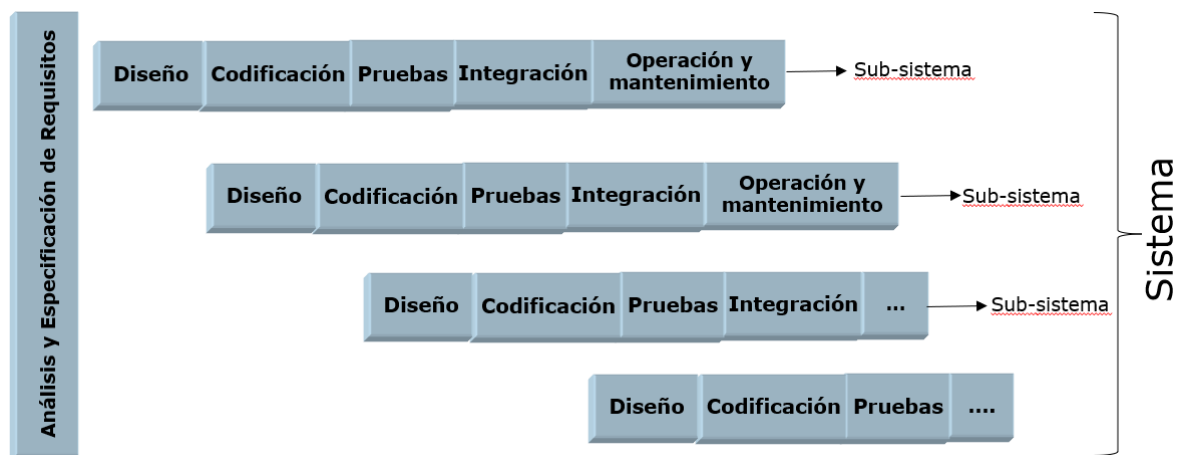
Se muestra como un espiral sobre ejes cartesianos que comienza en el primer cuadrante cerca del punto de cruce de los ejes y va creciendo con los giros de la espiral que pasan

por los distintos cuadrantes. En cada ciclo de la espiral se lleva a cabo una parte del desarrollo total que atraviesa las cuatro actividades.

Modelo incremental

El modelo incremental le suma flexibilidad al modelo de cascada, subsanando una de las grandes desventajas que este modelo poseía: no permitirle al cliente tener signos visibles del avance del proyecto. Parte de una especificación de requerimientos y descompone el desarrollo del sistema en una serie de partes, aplicándole un ciclo de desarrollo a cada una de ellas, y generando así subsistemas operativos que le permiten al cliente disponer del producto en forma incremental.

La figura que se muestra a continuación permite clarificar mejor el modelo:



Si bien en la gráfica se muestran solapamiento entre los ciclos, el inicio de cada uno depende del proyecto y la planificación que se lleve a cabo.

La generación de entregas parciales que este modelo permite realizar en períodos más cortos de tiempo, en comparación con el modelo de cascada, también habilita que los requisitos puedan irse completando o corrigiendo durante el desarrollo.

La definición de los subsistemas que se irán entregando requiere un poco más de experiencia del grupo de desarrollo para llevar a cabo dicha planificación, teniendo

presente que cada entrega debe ser una versión operativa que puedan emplear los usuarios.

Algunos autores plantean una modificación de este modelo llamado “Diseño por planificación”, donde las entregas están fijadas de acuerdo a la prioridad que tenga el cliente por los requerimientos que desea cumplimentar en orden decreciente, de forma tal que, si el proyecto por alguna razón debe ser cancelado o demorado, los requisitos más prioritarios ya hayan sido tenidos en cuenta e implementados con los primeros subsistemas operativos que recibió.

El modelo incremental puede ser muy apropiado cuando el cliente necesita disponer al menos de una parte de la funcionalidad del sistema en un tiempo corto, y también colabora mucho en calmar la ansiedad del cliente cuando desea ver resultados de la inversión que está haciendo en forma rápida.

Modelo de prototipos

Un prototipo es un modelo que trata de simular el funcionamiento que debería tener el sistema. Es de gran utilidad cuando el cliente no puede describir claramente que desea del sistema, o cuando el desarrollador desconoce las actividades que lleva a cabo la organización.

Originalmente, para comprender estos requerimientos del cliente, algunos desarrolladores optaban por generar un software piloto o “borrador” que se centraba en comprender la interfaz del sistema y podía, a veces, llegar a cubrir alguna mínima funcionalidad. Este “borrador” del sistema era un prototipo descartable que, si bien mostraba la interfaz o incluso funcionalidades, muchas veces no se había generado en el lenguaje de programación que pudiera ser más acorde a las necesidades del cliente. Así, una vez que las entradas, salidas y procesos del sistema estaban bien definidos, se descartaba el prototipo y se procedía al desarrollo del producto en la arquitectura definida.

Esta situación generaba un conflicto con el cliente porque no entendía que se descartara el trabajo ya hecho o, en ocasiones, era el mismo desarrollador quien luego no quería comenzar a trabajar en esa interfaz nuevamente. Esto dio lugar a cambios en el plan original para desarrollar el prototipo sobre la arquitectura elegida para el funcionamiento del sistema. De esta manera, el prototipo pasó a ser evolutivo (no se descartaría lo realizado anteriormente).

Los prototipos pueden categorizarse en:

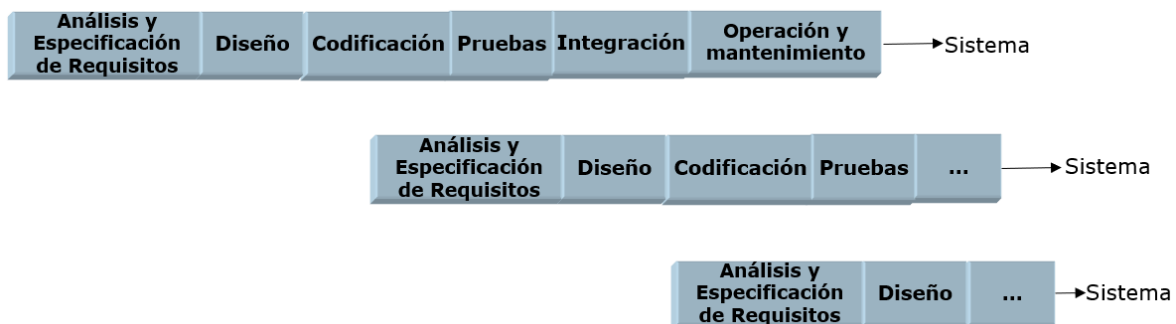
- **Ligeros:** se llevan a cabo bocetos de las pantallas, simulando los enlaces con dibujos de navegación.
- **Operativos:** se crean los módulos del software, cubriendo partes de las funcionalidades requeridas del sistema.

Es importante no perder de vista que el objetivo principal de este modelo es poder trabajar con el cliente y/o los usuarios para obtener una retroalimentación que ayude a definir los requisitos del sistema.

Modelo evolutivo

El modelo evolutivo está conformado por varios ciclos de desarrollo, donde se enlazan las actividades correspondientes a la especificación, el desarrollo y la validación, y cada uno genera un sistema completo que el cliente puede operar.

Veamos la gráfica del modelo:



Este sistema parte del desarrollo en forma rápida de un sistema inicial a partir de una lista de especificaciones, que se irá refinando mediante las nuevas peticiones del cliente para generar un sistema que pueda satisfacer todas sus necesidades.

Así, a medida que el cliente opere la versión entregada, surgirá la información para el nuevo desarrollo, donde se ampliarán los requisitos iniciales y se generará el diseño de la próxima versión.

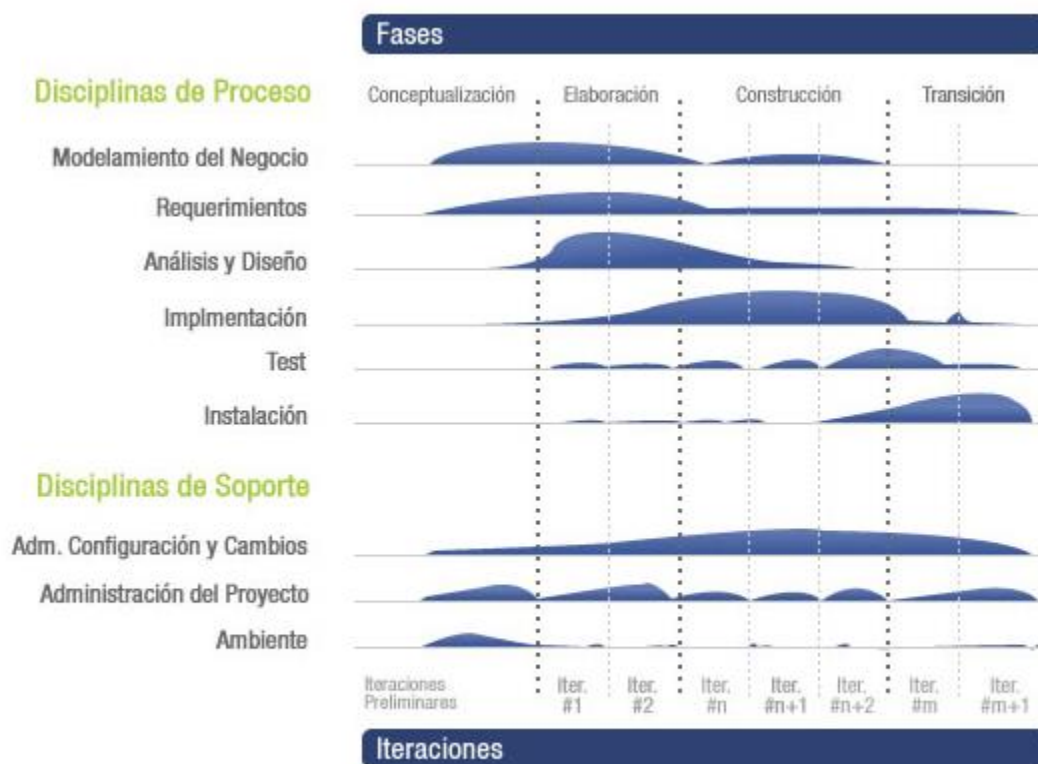
En la práctica, este modelo puede resultar útil si hay un desconocimiento sobre cuáles son las necesidades que debe cubrir el nuevo sistema. Esto puede suceder cuando no se posee experiencia previa en el área; se trató de un nuevo entorno para dicho sistema; se

requiera un sistema con extrema urgencia y no haya suficiente tiempo para identificar detalladamente todos los posibles requerimientos, o cuando el sistema operara en un entorno cambiante (por ejemplo, sujeto a normas jurídicas que aún se encuentran en proceso de definición).

Modelo UP (Proceso Unificado)

El Proceso Unificado de Rational, también conocido como *Rational Unified Process (RUP)*, está asociado a Lenguaje Unificado de Modelador (UML) y surge del Proceso Unificado de Desarrollo de Software que se puede aplicar con él, propuesto por sus autores, Ivar Jacobson, Grady Booch y James Rumbaugh. Se trata de un claro ejemplo de un modelo híbrido.

Veamos en la siguiente figura la gráfica del modelo:



El Proceso Unificado de Rational (RUP)

Fuente: IBM. El proceso unificado de rational (RUP). IBM Software Group

Este modelo define cuatro fases representadas en columnas. Cada una concluye con el cumplimiento de una serie de objetivos específicos, un entregable, que puede ser un modelo o documentación.

Las fases son:

- **Inicio:** se establecen los procesos de negocio, se identifica con quien interactuará el sistema y las interacciones.
- **Elaboración:** busca comprender el dominio del problema, generar el marco arquitectónico, establecer el plan de proyecto y los riesgos del mismo. Finaliza con la obtención de un modelo de requerimientos del sistema (con los casos de uso), la descripción de la arquitectura y el plan para el desarrollo del software.
- **Construcción:** abarca el diseño del sistema, su programación y pruebas. Finaliza con un sistema operativo y su documentación.
- **Transición:** el sistema pasa del ámbito de desarrollo a trabajar en el entorno real. Finaliza cuando el sistema de software se encuentra operando en el entorno operativo y funcionando correctamente con su documentación.

A su vez, cada una de estas fases, dependiendo de su complejidad asociada al proyecto del que se trate, puede ser dividida en iteraciones, cada una de las cuales representa un incremento.

Así mismo, cada iteración pasa por una serie de flujos de trabajo, representada en la primera columna de la gráfica en forma de filas. Esas fases se corresponden con las etapas que veníamos analizando en el modelo de cascada y cubren desde los requisitos, el análisis, diseño, implementación y pruebas.

El proceso unificado es un modelo iterativo e incremental, integrado por fases, ciclos, y flujos de trabajo.